



中国科学院大学
University of Chinese Academy of Sciences

Deep Learning

Regularization and optimization

Xinfeng Zhang (张新峰)

School of Computer Science and Technology

University of Chinese Academy of Sciences

Email: xfzhang@ucas.ac.cn



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



提纲

- 深度学习模型的训练与测试
- 参数范数正则化
- 数据增强
- Bagging
- 提前终止
- Dropout和DropConnect
- 归一化与优化

正则化与优化

□ 机器学习中的正则化方法可分为两类：狭义正则化和广义正则化。

- 狭义正则化是指通过在模型的损失函数中增加惩罚项来增强模型泛化能力的方法。
- 广义正则化是指可以减小模型过拟合并使得泛化误差显著降低的所有方法。

□ 本章讲述的是广义正则化，主要包括参数范数正则化、数据增强、Bagging、提前终止、Dropout和归一化等。



优化

- ❑ 优化（Optimization）是指在给定约束的情况下求解目标函数的最大值或最小值。
- ❑ 在深度学习中，深度神经网络的结构复杂、参数多，普遍存在梯度消失或梯度爆炸问题，优化起来非常困难，进而很难找到全局最优解。因此，深度学习算法需要选用合适的优化算法，才能使目标函数收敛。
- ❑ 本章讲述的深度学习中的优化算法主要包括常见的梯度下降法和基于动量的方法。



优化

- 深度学习模型的泛化性能非常重要，这意味着即使通过优化算法找到了训练集的近似最优解，仍然可能在测试集上得到很差的结果，即产生了过拟合。因此，在深度学习中，正则化与优化是需要结合起来使用的。





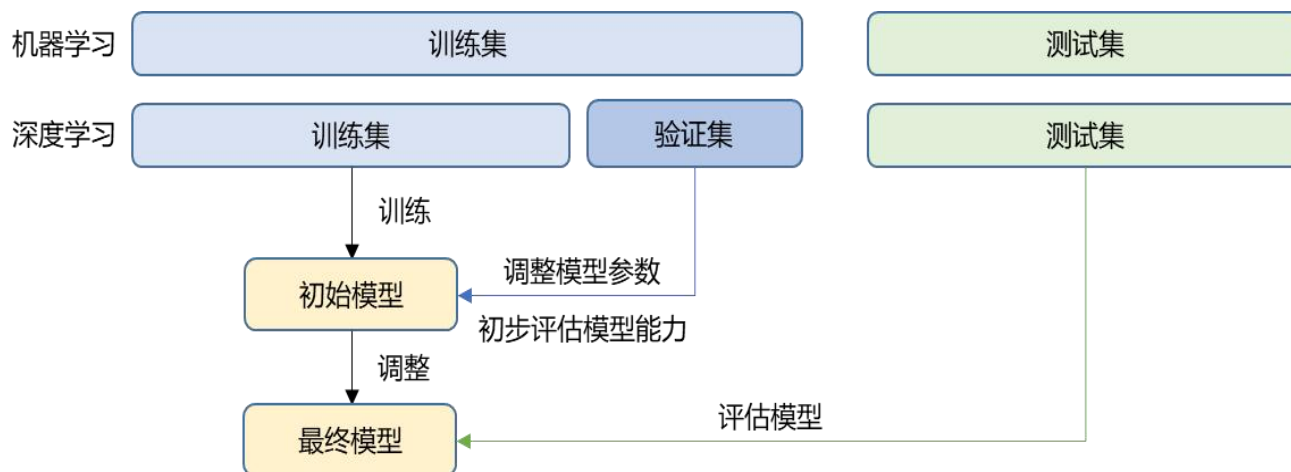
1

深度学习模型的训练与测试

深度学习中的数据划分

□ 数据集分类

- 训练集(Training set): 用于模型拟合的数据样本
- 验证集(Validation set): 是模型训练过程中单独留出的样本集, 它可以用于调整模型的超参数和用于对模型的能力进行初步评估
 - 例如SVM中参数 c 和核函数的选择, 或者选择网络结构
- 测试集(Test set): 用来评估最终模型的泛化能力。但不能作为调参、选择特征等算法相关的选择的依据



过拟合和欠拟合

□ 过拟合

- 是指模型能很好地拟合训练样本，而无法很好地拟合测试样本的现象，从而导致泛化性能下降
- 为防止“过拟合”，可以选择减少参数、降低模型复杂度、正则化等

□ 欠拟合

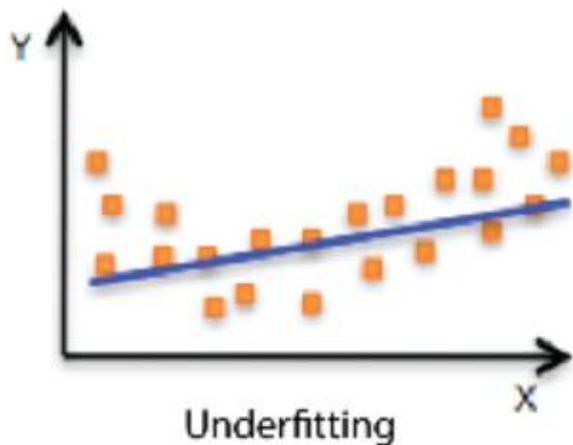
- 是指模型还没有很好地训练出数据的一般规律，模型拟合程度不高的现象
- 为防止“欠拟合”，可以选择调整参数、增加迭代深度、换用更加复杂的模型等



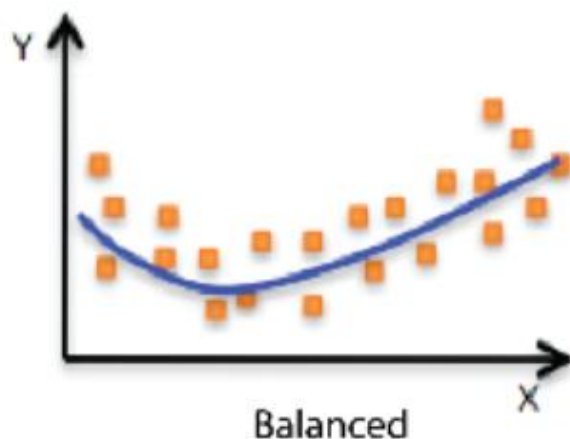
误差分析

□ 模型训练的状态

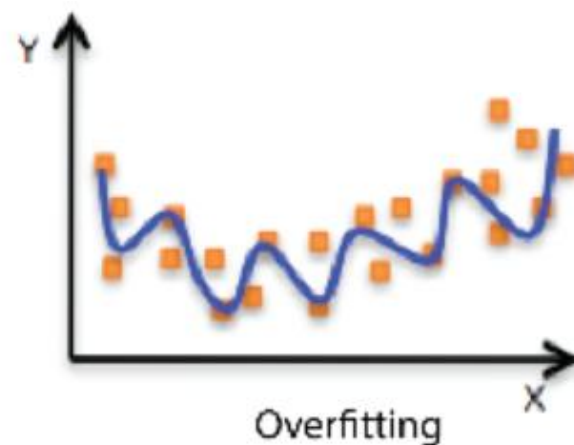
— 欠拟合、合适和过拟合的模型



训练集和测试集误差都比较大



训练集和测试集误差都比较小



训练集误差小
测试集误差大



数据集

- 给定数据集 $\mathbf{D}$ ，其中 D_{train} 是训练集， D_{test} 是测试集。
- 训练集 D_{train} 中的输入样本集合为 $\mathbf{X}_{train} = \{x_i\}(i = 1, 2, \dots, n)$ ，对应的标签值集合为 $\mathbf{L}_{train} = \{l_i\}(i = 1, 2, \dots, n)$ ，对应的为真实值集合 $\mathbf{Y}_{train} = \{y_i\}(i = 1, 2, \dots, n)$ 。
- 测试集 D_{test} 中的输入样本集合为 $\mathbf{X}_{test} = \{x_j\}(j = 1, 2, \dots, m)$ ，对应的标签值集合为 $\mathbf{L}_{test} = \{l_j\}(j = 1, 2, \dots, m)$ ，对应的真实值集合为 $\mathbf{Y}_{test} = \{y_j\}(j = 1, 2, \dots, m)$ 。
- 假定基于现有的训练集 D_{train} ，通过留出法获得样本数量相同的训练集 $\{D_k\}(k = 1, 2, \dots, K)$ ， $f_k(x; D_k)$ 表示在第 k 个训练集 D_k 上学习得到的模型。



偏差

- 不管是在传统机器学习还是深度学习中，偏差（Bias）度量训练所得模型的预估值和真实值的偏离程度，刻画了算法本身的拟合能力。
- 用 $(\hat{y}_j)_k = f_k(x_j; D_k)$ 表示在第 k 个训练集 D_k 上训练所得的模型对测试样本 x_j 的预估值输出，用 $\bar{y}_j$ 表示在 K 个训练集上训练所得的 K 个模型对测试样本 $x_j (j = 1, 2, \dots, m)$ 的预估值输出 $(\hat{y}_j)_k (k = 1, 2, \dots, K)$ 的期望，计算公式：

$$\bar{y}_j = E[\{(\hat{y}_j)_k\}] = E[\{f_k(x_j; D_k)\}], \quad k = 1, 2, \dots, K \quad (1)$$

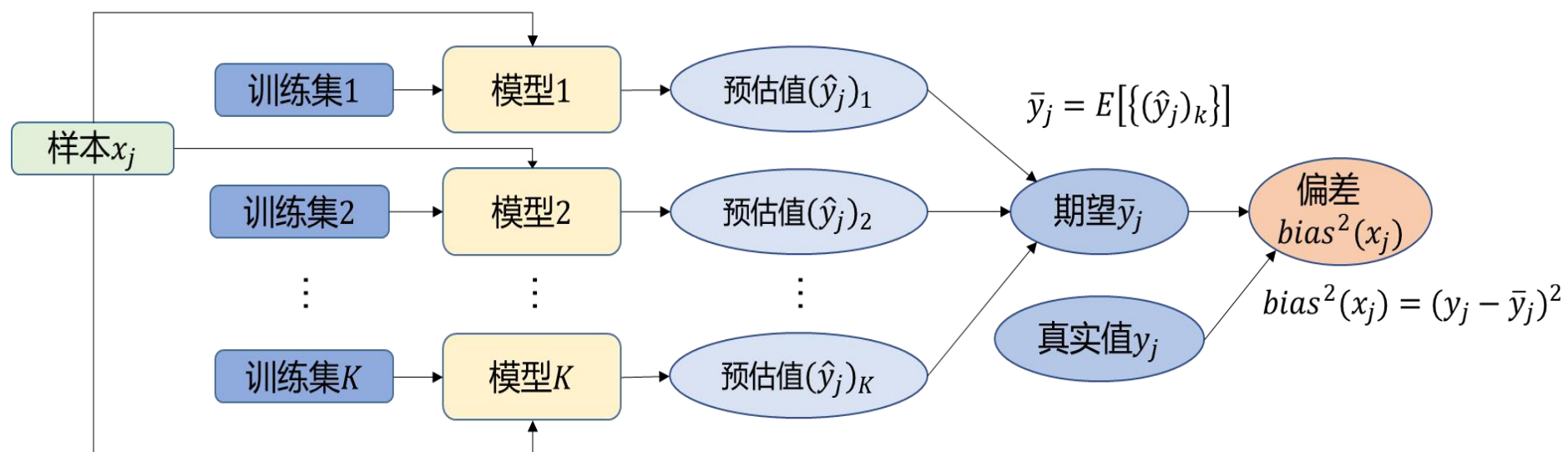
- 那么样本 x_j 的偏差计算公式：

$$bias^2(x_j) = (y_j - \bar{y}_j)^2$$



偏差

□ 偏差的计算过程如图所示



方差

- 方差 (Variance) 度量样本数量相同、但样本存在差异的不同训练集所导致的算法性能的变化，刻画了数据扰动对算法的影响。
- 同样，用 $(\hat{y}_j)_k = f_k(x_j; D_k)$ 表示在第 k 个训练集 D_k 上训练所得的模型对测试样本 x_j 的预估值输出，用 $\bar{y}_j$ 表示在 K 个训练集上训练所得的 K 个模型对测试样本 $x_j (j=1, 2, \dots, m)$ 的预估值输出 $(\hat{y}_j)_k (k = 1, 2, \dots, K)$ 的期望，计算公式：

$$\bar{y}_j = E[\{(\hat{y}_j)_k\}] = E[\{f_k(x_j; D_k)\}], k = 1, 2, \dots, K$$

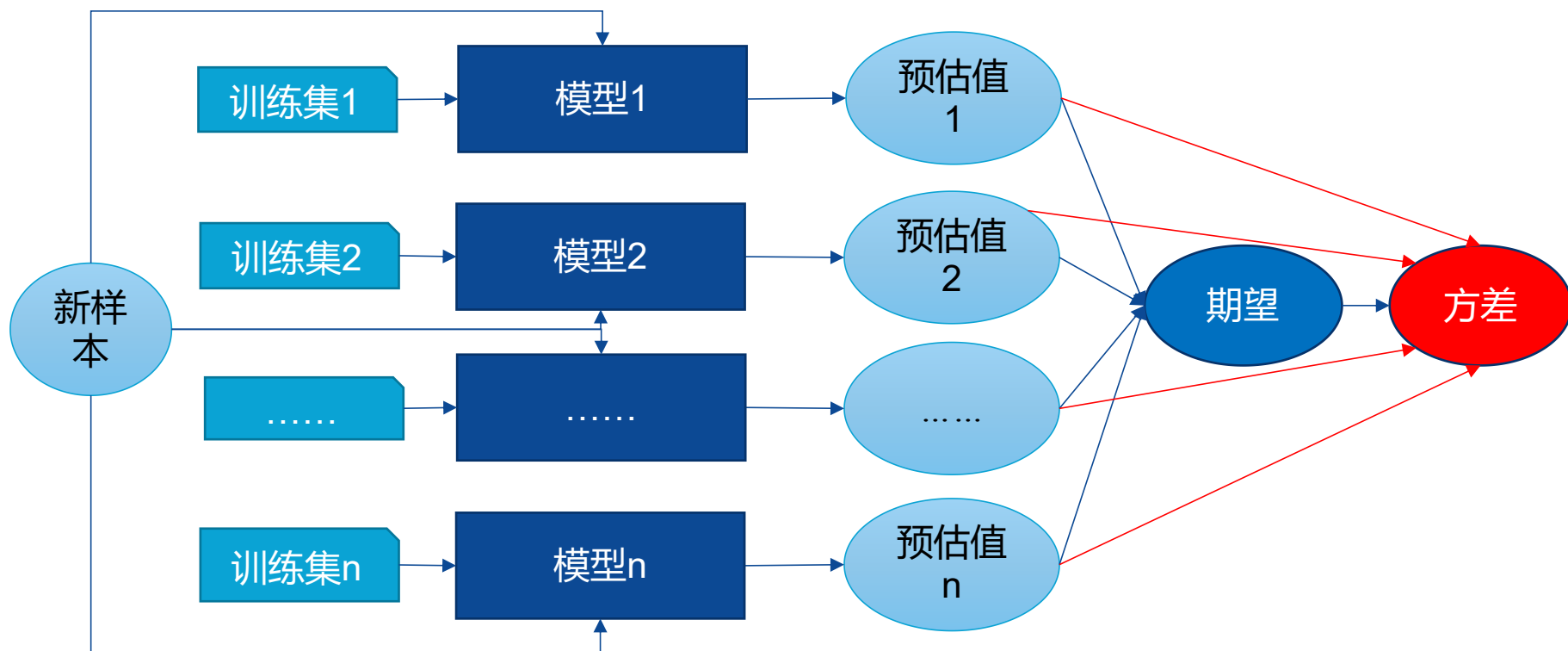
- 那么样本 x_j 的方差计算公式：

$$var(x_j) = E\left[\left\{((\hat{y}_j)_k - \bar{y}_j)^2\right\}\right], k = 1, 2, \dots, K$$



方差

□ 方差的计算过程如图所示



噪声

- 噪声 (Noise) 度量样本真实值与标签值的偏离程度，刻画了学习问题本身的难度。
- $y_j (j = 1, 2, \dots, m)$ 表示真实值, $l_j (j = 1, 2, \dots, m)$ 表示标签值。
- 单个样本 x_j 的噪声计算公式:

$$\varepsilon^2 = (l_j - y_j)^2$$

- 多个样本的噪声计算公式:

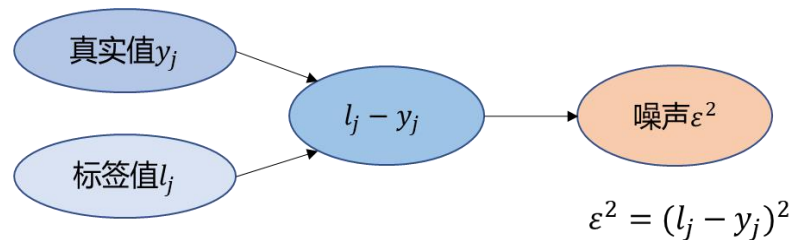
$$\varepsilon^2 = E[\{(l_j - y_j)^2\}], j = 1, 2, \dots, m$$



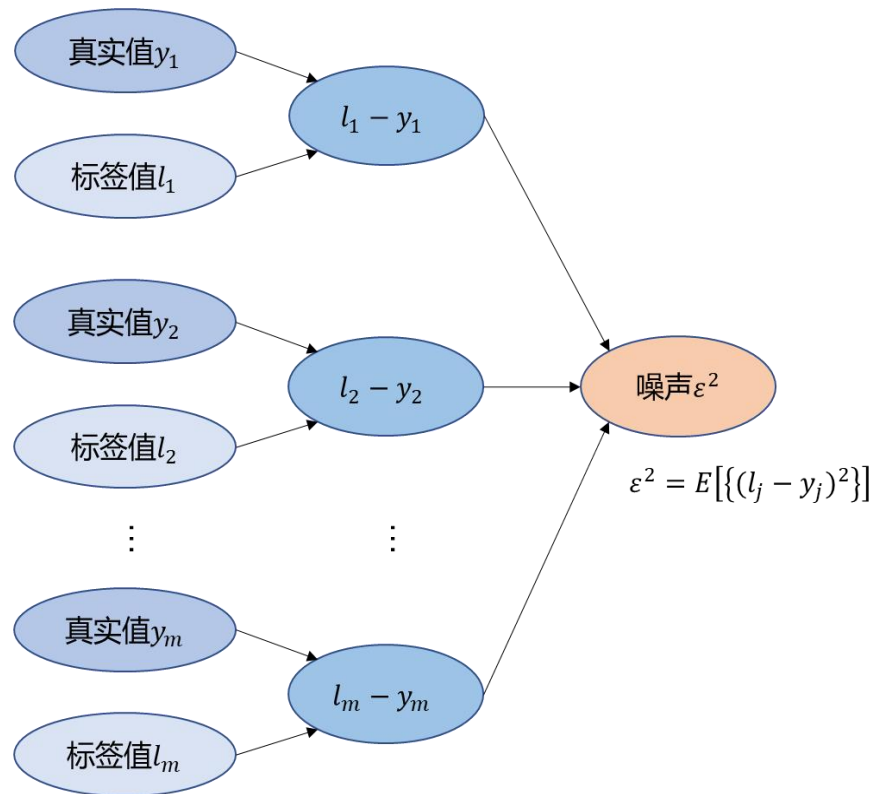
噪声

□ 噪声的计算过程如图所示

单样本的噪声



多样本的噪声



泛化误差

□ 泛化误差 (Generalization Error)

- 度量训练所得模型在总体数据上得到的预估值和标签值偏离程度的期望

泛化误差可分解为：偏差+方差



举例说明

- ❑ ①、④两种情况的训练集误差都很小，这种就称为low bias，说明训练的很到位了
- ❑ ②、③两种情况的训练集误差很大，这就称为high bias

	①	②	③	④
Train set(训练集) error	1%	15%	15%	0.8%
Dev set (验证集) error	11%	16%	30%	1%

(假设human performance为0%)



举例说明

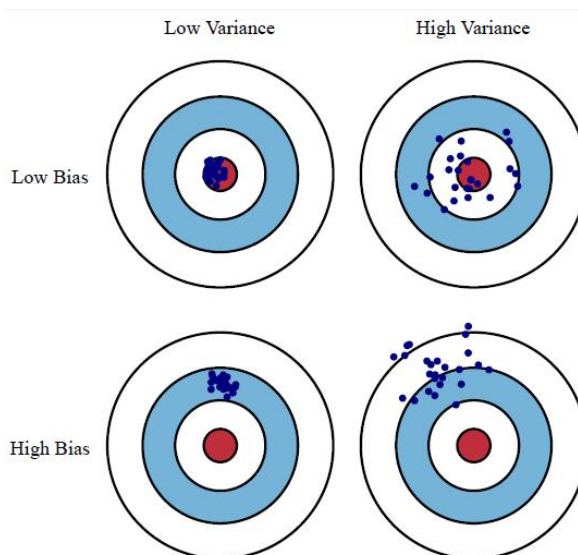
- ❑ ①情况下，验证集相比训练集误差上升了很多，这就是 **high variance**
- ❑ ②情况下，虽然它的验证集误差更大，但是相比它的训练集误差，基本没太大变化
- ❑ ④情况下，验证集误差相比训练集误差上升很小，我们说是 **Low variance**

	1	2	3	4
Train set(训练集) error	1%	15%	15%	0.8%
Dev set (验证集) error	11%	16%	30%	1%

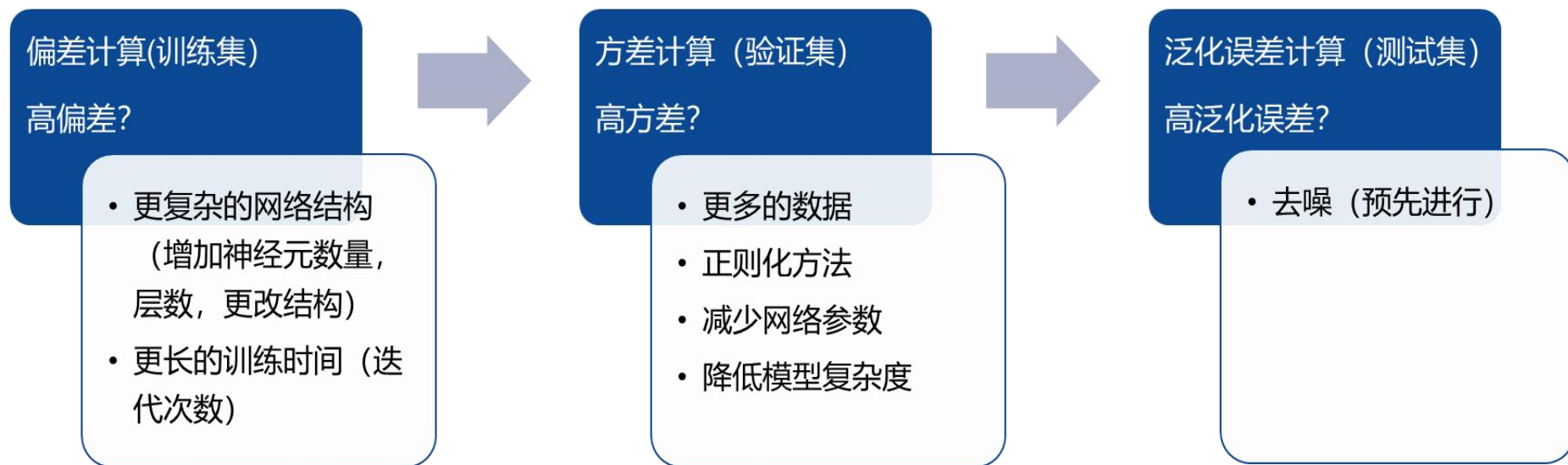
(假设human performance为0%)



举例说明



深度学习模型的训练与测试过程





2

参数范数正则化

正则化概念

□ 好的模型

- 在训练数据上表现好？（基本要求）
- 且在新输入（测试数据）上有较好的泛化性。（重点）

□ 正则化（Regularization）

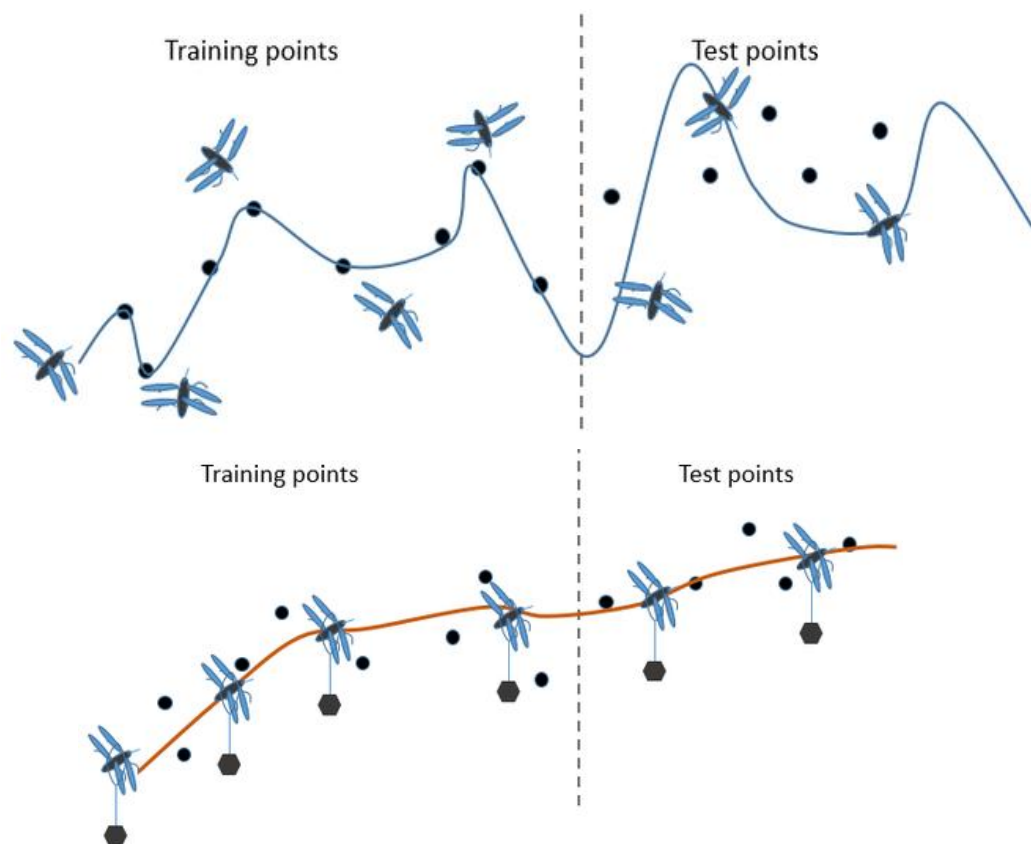
- 广义正则化：通过某种手段使学习算法在训练误差变化不大的情况下，使得泛化误差显著降低的方法
- 狭义正则化：不减少网络参数，只进行参数范围调整的方法



正则化概念（举例说明）

□ 正则化相当于“挂坠”

□ 狭义正则化就是“给损失函数加一个正则化项（挂坠）”



范数

- 机器学习中，很多时候都需要衡量一个向量的大小，此时便需要用到范数的知识
- 范数是将向量映射到非负值的函数，它满足三条性质：
 - 非负性 $f(x) \geq 0 (f(x) = 0 \Leftrightarrow x = 0)$
 - 齐次性 $\forall \alpha \in R, f(\alpha x) = |\alpha| f(x)$
 - 三角不等式 $f(x+y) \leq f(x) + f(y)$

范数

- 常用范数： L_p 范数是使用最为广泛的一种范数， $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$ ，其中 $p=2$ 时，该范数等价于向量和原点的欧几里得距离
- 有时候也需要衡量矩阵的大小，在深度学习中，最常见的做法便是使用Frobenius范数(F范数)，即 $\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}$

参数范数正则化

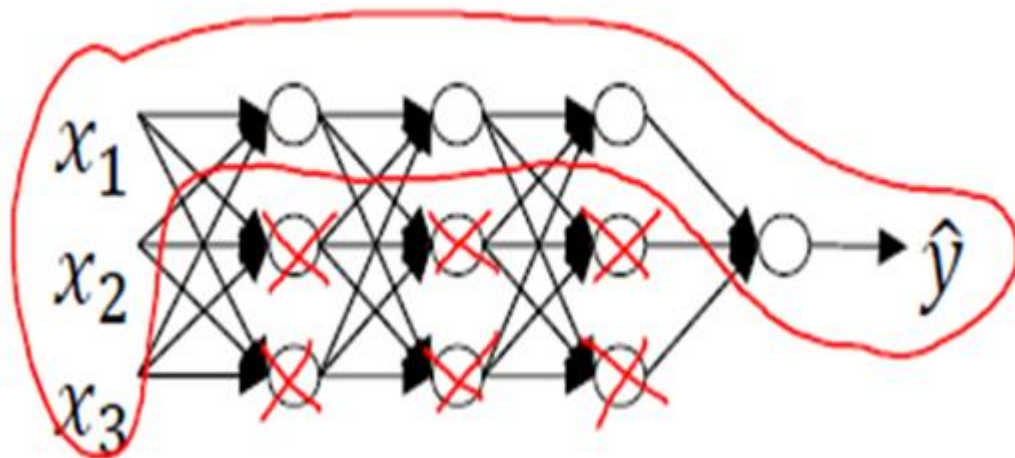
- 通过简单地在损失函数 $J(\cdot)$ 后添加一个参数范数正则化项 $\Omega(\theta)$ ，来限制模型的学习能力，正则化后的损失函数可表示为 $\tilde{L}$:

$$\tilde{L}(\theta; x, y) = L(\theta; x, y) + \alpha\Omega(\theta)$$

- 其中， α 是正则项系数，它是一个超参，用来权衡惩罚项 Ω 对损失函数的贡献。若 α 为0，表示无正则化； α 越大，正则化惩罚越大。
- 我们希望正则化后的训练误差（第一项，蓝色）最小，又希望模型尽量简单（第二项，红色）

参数范数正则化

- 当最小化正则化后的损失函数时候，同时降低原有损失函数 $L(\cdot)$ 和正则化项 $\Omega(\theta)$ 的值，一定程度减少了过拟合的程度
 - 通俗来说，正则化就是让参数（如权重 w ）多几个等于0或者接近于0，说明该节点影响很小，如果是神经网络相当于在神经网络中删掉了一些节点，这样模型就变简单了
- 对于参数范数正则化项，可以选择不同的范数形式，得到不同的优先解。



参数范数正则化

□ 为了让参数多几个为0，对于正则化项 Ω ，定义如下3种范数

➤ L0范数： $\|\mathbf{W}\|_0$ ，指向量中非0的元素的个数，越小说明0元素越多

➤ L1范数： $\|\mathbf{W}\|_1$ ，指向量中各个元素绝对值之和

➤ L2范数： $\|\mathbf{W}\|_2$ ，即各元素的平方和再开方

L2 正则化

- L2正则化主要用于线性回归，又称为岭回归(Ridge Regression) 或权重衰减(Weight decay)，添加的正则化项形式如下：

$$\Omega(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|_2^2$$

- 损失函数

$$\tilde{L}(\mathbf{w}; \mathbf{x}, \mathbf{y}) = L(\mathbf{w}; \mathbf{x}, \mathbf{y}) + \frac{\alpha}{2} \|\mathbf{w}\|_2^2$$

L2 正则化

□ L2参数正则化分析（单步）

- 对于某一模型，假设要对其所有的参数进行参数正则化，则正则化后的整体损失函数 $\tilde{J}$ 如下

$$\tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \frac{\alpha}{2} \mathbf{w}^T \mathbf{w}$$

- 计算梯度

$$\nabla_{\mathbf{w}} \tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \nabla_{\mathbf{w}} L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \alpha \mathbf{w}$$

L2 正则化

□ L2参数正则化分析（单步）

- 使用单步梯度下降更新权重：

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} - \lambda(\nabla_{\mathbf{w}}L(\mathbf{w}; X, \mathbf{y}) + \alpha\mathbf{w}) \\ &= (1 - \lambda\alpha)\mathbf{w} - \lambda\nabla_{\mathbf{w}}L(\mathbf{w}; X, \mathbf{y})\end{aligned}$$

- 在进行每步梯度更新前，先对参数 $\mathbf{w}$ 进行了缩放。因此，这也是权重衰减名称的来源

L2 正则化

□ L2参数正则化分析（整个训练过程）

- $\mathbf{w}^*$: 未正则化的损失函数 L 得到的最小损失权重向量, 即:

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} L(\mathbf{w})$$

- 在 $\mathbf{w}^*$ 附近对损失函数做二次近似, 近似的 $\hat{L}(\mathbf{w})$ 如下:

$$\hat{L}(\mathbf{w}) = L(\mathbf{w}^*) + (\mathbf{w} - \mathbf{w}^*)^T \nabla L(\mathbf{w}^*) + \frac{1}{2} (\mathbf{w} - \mathbf{w}^*)^T \mathbf{H} (\mathbf{w} - \mathbf{w}^*)$$

- $\mathbf{H}$ 是 L 在 $\mathbf{w}^*$ 处计算的Hessian矩阵。当 $\hat{L}(\mathbf{w})$ 取得最小值时, 其梯度应为0

$$\nabla_{\mathbf{w}} \hat{L}(\mathbf{w}) = \mathbf{H}(\mathbf{w} - \mathbf{w}^*) = 0$$

$$\mathbf{H}(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

L2 正则化

□ L2参数正则化分析（整个训练过程）

- 引入L2正则化项的梯度，设变量 $\tilde{\mathbf{w}}$ 使得 $\hat{L}(\mathbf{w})$ 最小

$$\nabla_{\mathbf{w}} \hat{L}(\mathbf{w}) = \alpha \tilde{\mathbf{w}} + \mathbf{H}(\tilde{\mathbf{w}} - \mathbf{w}^*) = 0$$

- 求得：
$$\tilde{\mathbf{w}} = (\mathbf{H} + \alpha \mathbf{I})^{-1} \mathbf{H} \mathbf{w}^*$$

- 因为 $\mathbf{H}$ 是对称矩阵，可分解为 $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$ ，所以

$$\tilde{\mathbf{w}} = \mathbf{Q}(\mathbf{\Lambda} + \alpha \mathbf{I})^{-1} \mathbf{\Lambda} \mathbf{Q}^T \mathbf{w}^*$$

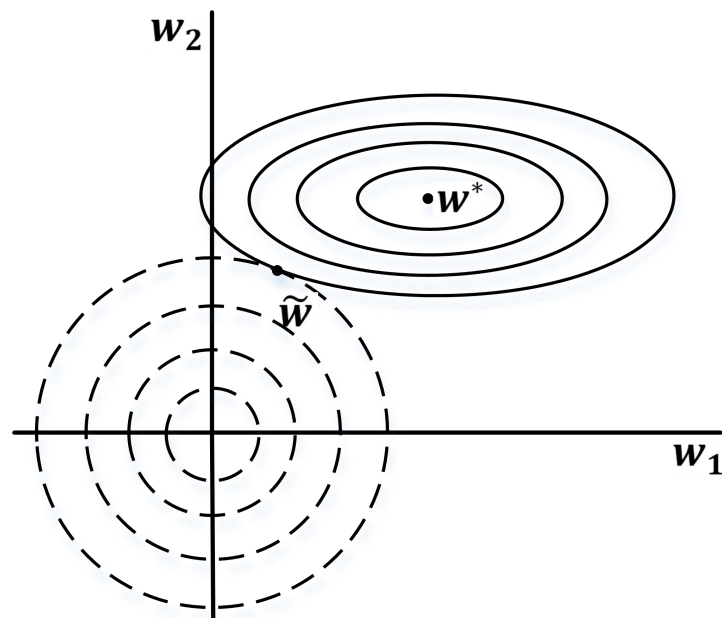
L2 正则化

□ L2参数正则化分析（整个训练过程）

$$\tilde{\mathbf{w}} = Q(\Lambda + \alpha I)^{-1}\Lambda Q^T \mathbf{w}^*$$

$$(\Lambda + \alpha I)^{-1}\Lambda = \begin{bmatrix} \frac{\lambda_1}{\lambda_1 + \alpha} & 0 & \dots & 0 \\ 0 & \frac{\lambda_2}{\lambda_2 + \alpha} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \frac{\lambda_n}{\lambda_n + \alpha} \end{bmatrix}$$

可见，权重衰减是沿着 $\mathbf{H}$ 的特征向量的标准正交基所定义的轴缩放 $\mathbf{w}^*$ 。当原损失函数在特征值大的方向进行下降时，正则化项对其影响较小；在特征值小的方向进行下降时，正则化项会对其进行限制。也就是说，保证了原损失函数沿着损失下降最大的方向下降



稀疏问题

- **特征选择**： x_i 的大部分元素（也就是特征）都是和最终的输出 y_i 没有关系；但在预测新的样本时，这些没用的信息反而会被考虑，从而干扰了对正确 y_i 的预测。稀疏规则化算子的引入就是为了完成**特征自动选择**，它会学习地去掉这些没有作用的特征，也就是把这些特征对应的**权重置为0**
- **可解释性**： 患病回归模型 $y=w_1*x_1+w_2*x_2+\dots+w_{1000}*x_{1000}+b$ ，通过学习，如果最后学习到的 w^* 就只有很少的非零元素，例如只有5个非零的 w_i ，也就是说，患不患这种病只和这5个因素有关，那医生就好分析多了

L1 正则化

□ L1正则化又称为Lasso回归（Lasso Regression），在损失函数中添加待正则化参数 w 的L1-范数（即 w 所有参数绝对值之和）作为正则化项

$$\Omega(w) = \|w\|_1$$

— 损失函数：

$$\tilde{L}(w; X, y) = L(w; X, y) + \alpha \|w\|_1$$

L1 正则化

□ L1参数正则化分析

$$\tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \alpha \|\mathbf{w}\|_1$$

– 计算梯度：

$$\nabla_{\mathbf{w}} \tilde{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \nabla_{\mathbf{w}} L(\mathbf{w}; \mathbf{X}, \mathbf{y}) + \alpha \cdot \text{sign}(\mathbf{w})$$

– 其中， $\text{sign}(\mathbf{w})$ 只是简单的取 $\mathbf{w}$ 各个元素的正负号。L1正则化与L2不大一样，不一定能得到 $L(\mathbf{w}; \mathbf{X}, \mathbf{y})$ 二次近似的直接算术解

L1 正则化

□ L1参数正则化分析

- L1正则项在完全一般化的Hessian矩阵无法得到清晰的代数表达式，
进一步假设Hessian为对角矩阵，即 $\mathbf{H} = \text{diag}([H_{1,1}, \dots, H_{n,n}])$ ，且 $H_{i,i} > 0$ 。(如果线性回归问题中的数据已被预处理，去除输入特征之间的相关性，那么这一假设成立)
- 可将目标函数二次近似分解为关于参数的总和

$$\tilde{L}(\mathbf{w}) = L(\mathbf{w}^*) + \sum_i \left[\frac{1}{2} H_{i,i} (w_i - w_i^*)^2 + \alpha |w_i| \right]$$

$$\widetilde{w}_i = w_i^* - \text{sign}(w_i) \frac{\alpha}{H_{i,i}}$$

L1 正则化

□ L1参数正则化分析

- 最小化这个近似损失函数可得

$$\widetilde{w}_i = \text{sign}(w_i^*) \max \left\{ |w_i^*| - \frac{\alpha}{H_{i,i}}, 0 \right\}$$

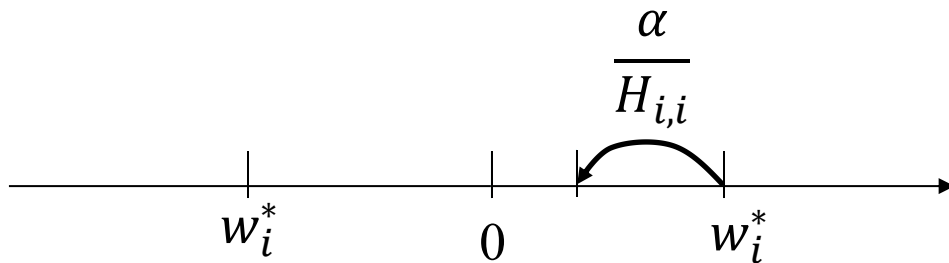
这里 w_i 和 w_i^* 符号相同

- $w_i^* > 0$ 时分两种情况

- $w_i^* \leq \frac{\alpha}{H_{ii}}; \widetilde{w}_i = 0$
- $w_i^* > \frac{\alpha}{H_{ii}}; \widetilde{w}_i = w_i^* - \frac{\alpha}{H_{i,i}}$

- $w_i^* < 0$ 时分两种情况

- $w_i^* \leq -\frac{\alpha}{H_{ii}}; \widetilde{w}_i = w_i^* + \frac{\alpha}{H_{i,i}}$
- $w_i^* > -\frac{\alpha}{H_{ii}}; \widetilde{w}_i = 0$



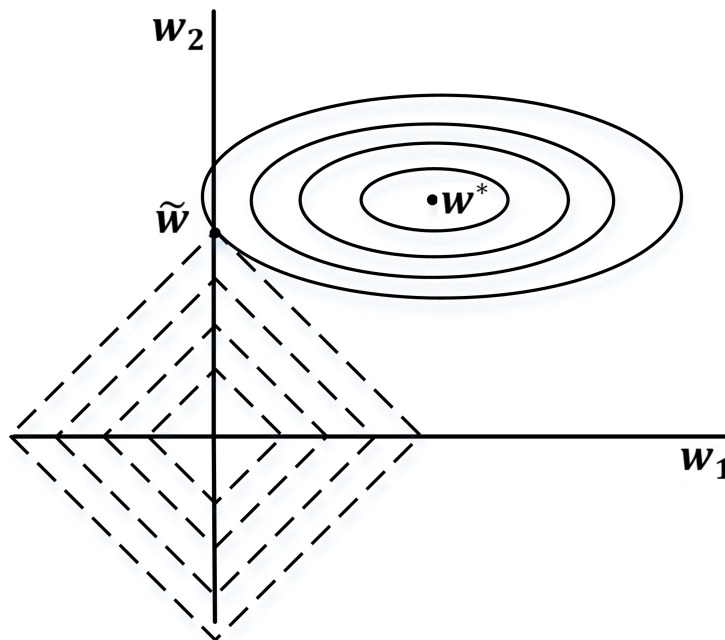
L1 正则化

□ L1参数正则化分析

- 最小化这个近似损失函数可得

$$\tilde{w}_i = \text{sign}(w_i^*) \max \left\{ |w_i^*| - \frac{\alpha}{H_{i,i}}, 0 \right\}$$

- 得到的参数 $\tilde{w}$ 的各个元素可能为0。因此，与L2正则化相比，得到的参数更加**稀疏**



L1 与L2的区别

- L2范数更有助于计算病态的问题
- L1相对于L2能够产生更加稀疏的模型
- 从概率角度进行分析，很多范数约束相当于对参数**添加先验分布**，其中L2范数相当于参数服从高斯先验分布；L1范数相当于拉普拉斯分布

$$\begin{aligned}\operatorname{argmax}_{\mathbf{w}} p(\mathbf{w}|X) &= \operatorname{argmax}_{\mathbf{w}} \frac{p(X|\mathbf{w})p(\mathbf{w})}{p(X)} = \operatorname{argmax}_{\mathbf{w}} p(X|\mathbf{w})p(\mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} \left(\prod_{i=1}^n p(x_i|\mathbf{w}) \right) p(\mathbf{w})\end{aligned}$$

$$\operatorname{argmax}_{\mathbf{w}} L(\mathbf{w}) = \prod_{i=1}^n \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y_i - \mathbf{w}^T \mathbf{x}_i)^2}{2\sigma^2}\right) \prod_{j=1}^d \frac{1}{\tau\sqrt{2\pi}} \exp\left(-\frac{(\mathbf{w}_j)^2}{2\tau^2}\right)$$

$$\begin{aligned}\operatorname{argmax}_{\mathbf{w}} L'(\mathbf{w}) &= \ln \prod_{i=1}^n \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y_i - \mathbf{w}^T \mathbf{x}_i)^2}{2\sigma^2}\right) \prod_{j=1}^d \frac{1}{\tau\sqrt{2\pi}} \exp\left(-\frac{(\mathbf{w}_j)^2}{2\tau^2}\right) \\ &= -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 - \frac{1}{2\tau^2} \sum_{j=1}^d (\mathbf{w}_j)^2 + \dots\end{aligned}$$

稀疏表示

- 数据集可视为矩阵 S ，其中**矩阵的行数表示样本的数量，矩阵的列数则表示对应的特征**。在很多应用中，矩阵 S 中存在很多零元素，但这些零元素并不是以整列、整行形式存在
 - 如在文档分类领域，我们将每一篇文档作为一个样本，那么数据集可表示为每一行表示为一个文档，每一列为一个字（词），行列交汇处就是某字（词）在某文档中出现的频率

稀疏表示

- 如果矩阵 S 是普通的非稀疏矩阵，稀疏表示目的是为这个普通稠密表达的样本 S 找到一个合适的字典，将样本转换为**稀疏表示 (Sparse Representation) 形式**，从而在学习训练的任务上得到简化，降低模型的复杂度。亦被称作**字典学习 (Dictionary Learning)**
 - 稀疏表示的本质是使用尽可能少的数据表示尽可能多的特征，并且同时也能带来计算效率的提升

稀疏表示学习

□ 给定数据集为 $\{x_1, x_2, \dots, x_n\}$, 字典学习可表示为

$$\min_{B, \alpha_i} \sum_{i=1}^m \|x_i - B\alpha_i\|_2^2 + \lambda \sum_{i=1}^m \|\alpha_i\|_1$$

其中 $B \in \mathbb{R}^{d \times k}$, k 为字典的词汇量, $\alpha_i \in \mathbb{R}^k$ 则是样本 $x_i \in \mathbb{R}^d$ 的稀疏表示

□ 第一项保证能够通过字典 B 由稀疏表示 α_i 重构样本数据 x_i , 第二项保证 α_i 尽可能稀疏。学习任务是求解字典 B , 以及 x_i 的稀疏表示 α_i , 可采用变量交替优化的策略进行学习

稀疏表示学习

□ 变量交替优化的策略

- 第一步，固定字典 B ，按照LASSO解法求解下式，从而可为每一个样本 x_i 求得对应的 α_i

$$\min_{\alpha_i} \|x_i - B\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1$$

- 第二步，利用求得的 α_i 更新字典 B ，求解下式进行更新

$$\min_B \|X - BA\|_F^2$$

其中 $X = (x_1, x_2, \dots, x_m) \in \mathbb{R}^{d \times m}$ ， $A = (\alpha_1, \alpha_2, \dots, \alpha_m) \in \mathbb{R}^{k \times m}$ ， $\|\cdot\|_F$ 为矩阵的Frobenius范数

- 对字典矩阵 B 进行初始化之后反复迭代上述两步，最终则可求得字典 B 和样本 x_i 对应的稀疏表示 α_i



3

数据增强

数据增强

□ 数据增强(Data Augmentation)

- 可以简单的理解为对训练数据数量的扩充，利用已有的有限数据产生更多的有效数据。如图像空间变换、像素颜色变换。
- 原因：深度学习离不开大规模标签数据集的出现，如果数据量太少，极有可能使得模型陷入过拟合之中。
- 目的：一方面可以尽可能地避免过拟合；另一方面随着训练数据的增多，也会在一定程度上提高模型的泛化性。
- 实现：OpenCV、PIL及深度学习框架内嵌方法

数据增强

□ **数据增强(Data Augmentation):** 图像数据增强主要有两种常见的方法：空间变换和像素颜色变换。

- 空间变换，如对图像进行随机翻转、旋转、平移和裁剪等操作。
- 像素颜色变换是对图像像素值进行修改，如在原始图像上进行对比度扰动、饱和度扰动、颜色变换与噪声扰动等操作。
- 可以利用OpenCV、PIL等图像处理库进行数据增强，同时在常用的深度学习框架（如Tensorflow、PyTorch和飞桨）中，都有相应的图像增强函数可供直接调用。

数据增强方法

□ PyTorch中与图像数据增强相关的函数位于 `torchvision.transforms` 中，包括：

- 颜色变换函数 `ColorJitter(brightness=0, contrast=0, saturation=0, hue=0)`：可以对输入图像的亮度（brightness）、对比度（contrast）、饱和度（saturation）和色调（hue）进行随机变换。
- 灰度变换函数 `Grayscale(num_output_channels=1)`：直接将输入图像转换为灰度图像，channels=1是默认值，也可以指定其他值。
- 灰度变换函数 `RandomGrayscale(p=0.1)`：在概率p下将输入图像转换为灰度图像，p=0.1是默认值，也可以指定其他值。

数据增强方法

□ PyTorch中与图像数据增强相关的函数位于 `torchvision.transforms` 中，包括：

- 随机旋转变换函数 `RandomRotation(degrees)`：参数 `degrees` 用于指定可以旋转的度数范围。
- 随机水平翻转函数 `RandomHorizontalFlip(p=0.5)`：在概率 `p` 下将输入图像进行水平翻转，`p=0.5` 是默认值，也可以指定其他值。
- 随机垂直翻转函数 `RandomVerticalFlip(p=0.5)`：在概率 `p` 下将输入图像进行垂直翻转，`p=0.5` 是默认值，也可以指定其他值。
- 仿射变换函数 `RandomAffine(degrees, translate=None, scale=None, shear=None)`：对输入图像进行仿射变换，参数 `degrees` 指定旋转角度，`translate` 设置平移区间，`scale` 设置缩放比例，`shear` 设置错切角度。

数据增强

□ 基于图像处理的数据增强方法

- 示例：如在深度学习框架Keras中有一个专门ImageDataGenerator类可用于进行简单的数据增强，其参数如下

- featurewise_center: 将输入数据的均值设置为 0，逐特征进行
- featurewise_std_normalization: 将输入除以数据标准差，逐特征进行
- zca_whitening: 对输入数据施加ZCA白化
- rotation_range: 数据提升时图片随机转动的角度
- width_shift_range: 图片宽度的某个比例，数据提升时图片随机水平偏移的幅度
- height_shift_range: 图片高度的某个比例，数据提升时图片随机竖直偏移的幅度
- shear_range: 剪切强度(逆时针方向的剪切变换角度)
- horizontal_flip: 进行随机水平翻转
- vertical_flip: 进行随机竖直翻转
- zoom_range: 随机缩放范围。
- channel_shift_range: 随机通道转换的范围
- fill_mode: 填充新像素的模式
- ...

```
keras.preprocessing.image.ImageDataGenerator(featurewise_center=False,  
                                               samplewise_center=False,  
                                               featurewise_std_normalization=False,  
                                               samplewise_std_normalization=False,  
                                               zca_whitening=False,  
                                               zca_epsilon=1e-06,  
                                               rotation_range=0,  
                                               width_shift_range=0.0,  
                                               height_shift_range=0.0,  
                                               brightness_range=None,  
                                               shear_range=0.0,  
                                               zoom_range=0.0,  
                                               channel_shift_range=0.0,  
                                               fill_mode='nearest',  
                                               cval=0.0,  
                                               horizontal_flip=False,  
                                               vertical_flip=False,  
                                               rescale=None,  
                                               preprocessing_function=None,  
                                               data_format=None,  
                                               validation_split=0.0,  
                                               dtype=None)
```

数据增强

□ 基于图像处理的数据增强方法

- 示例：如在深度学习框架Keras中有一个专门ImageDataGenerator类可用于进行简单的数据增强，其参数如下

```
keras.preprocessing.image.ImageDataGenerator(featurewise_center=False,  
                                              sampleswise_center=False,  
                                              featurewise_std_normalization=False,  
                                              sampleswise_std_normalization=False,  
                                              zca_whitening=False,  
                                              zca_epsilon=1e-06,  
                                              rotation_range=0,  
                                              width_shift_range=0.0,  
                                              height_shift_range=0.0,  
                                              brightness_range=None,  
                                              shear_range=0.0,  
                                              zoom_range=0.0,  
                                              channel_shift_range=0.0,  
                                              fill_mode='nearest',  
                                              cval=0.0,  
                                              horizontal_flip=False,  
                                              vertical_flip=False,  
                                              rescale=None,  
                                              preprocessing_function=None,  
                                              data_format=None,  
                                              validation_split=0.0,  
                                              dtype=None)
```

preprocessing_function: 自行编写的图像处理函数，对于ImageDataGenerator没有实现的数据增强操作，也可以自行编写函数传入到preprocessing_function中

数据增强示例

原始图像



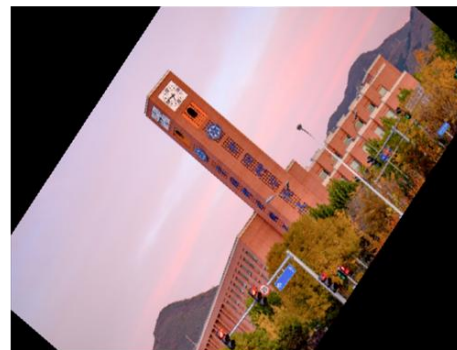
颜色变换



灰度变换



图像旋转



水平翻转



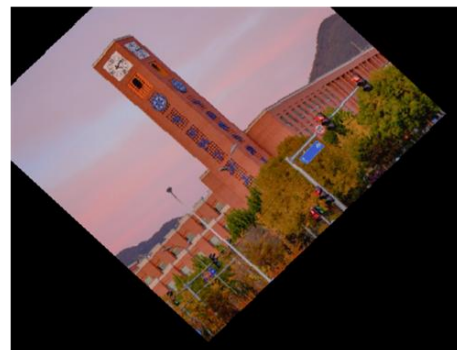
垂直翻转



仿射变换



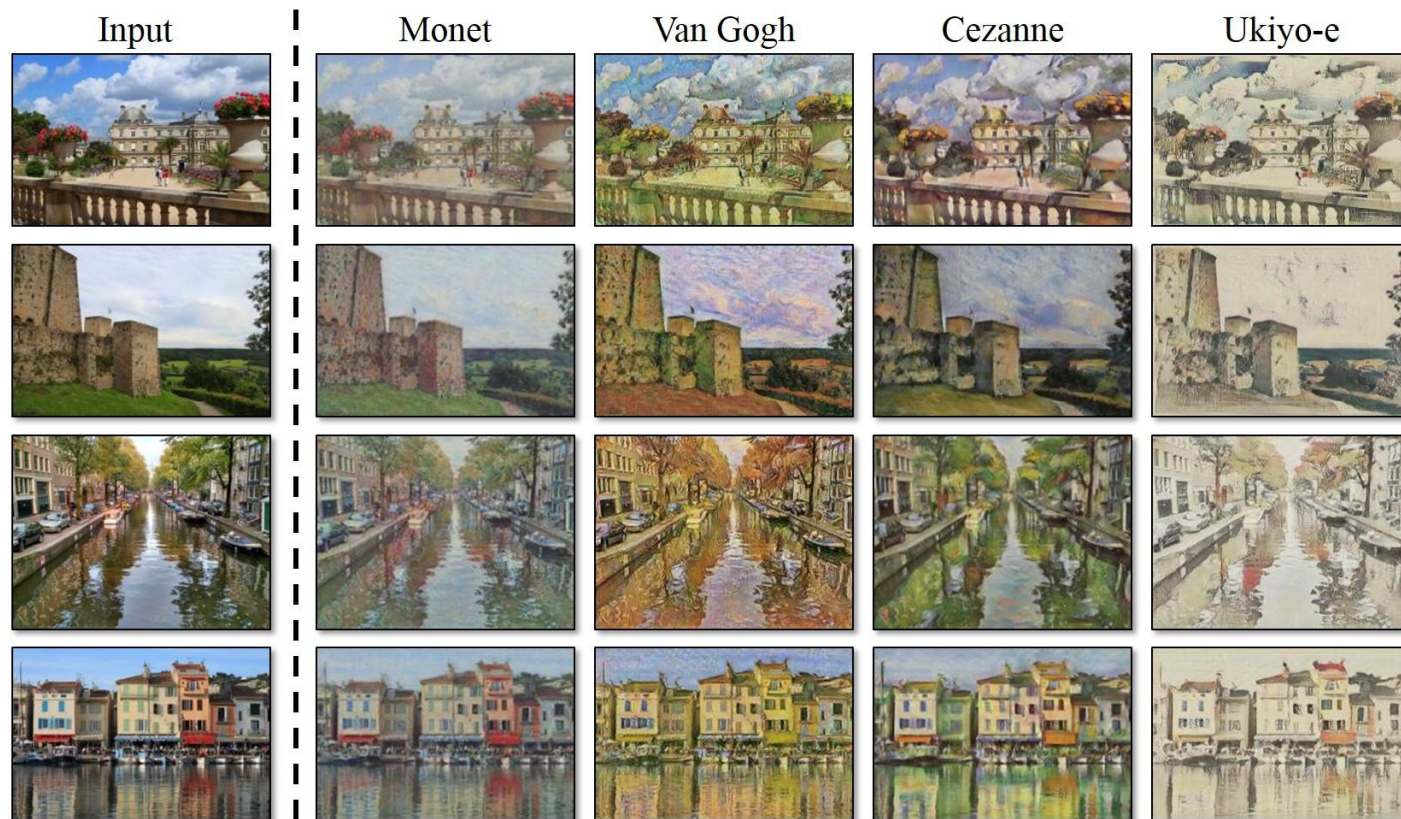
组合变换



数据增强

□ 基于深度学习的方法

- 对抗生成网络数据增强
- 风格转换网络数据增强



数据增强

- 需要指出的是，对数据进行数据增强也需要视情况而定。如在手写数字识别任务中，为了区别6和9的区别，也不适合对数据进行垂直翻转以及旋转过大角度

GitHub上数据增强的开源实现<https://github.com/aleju/imgaug>

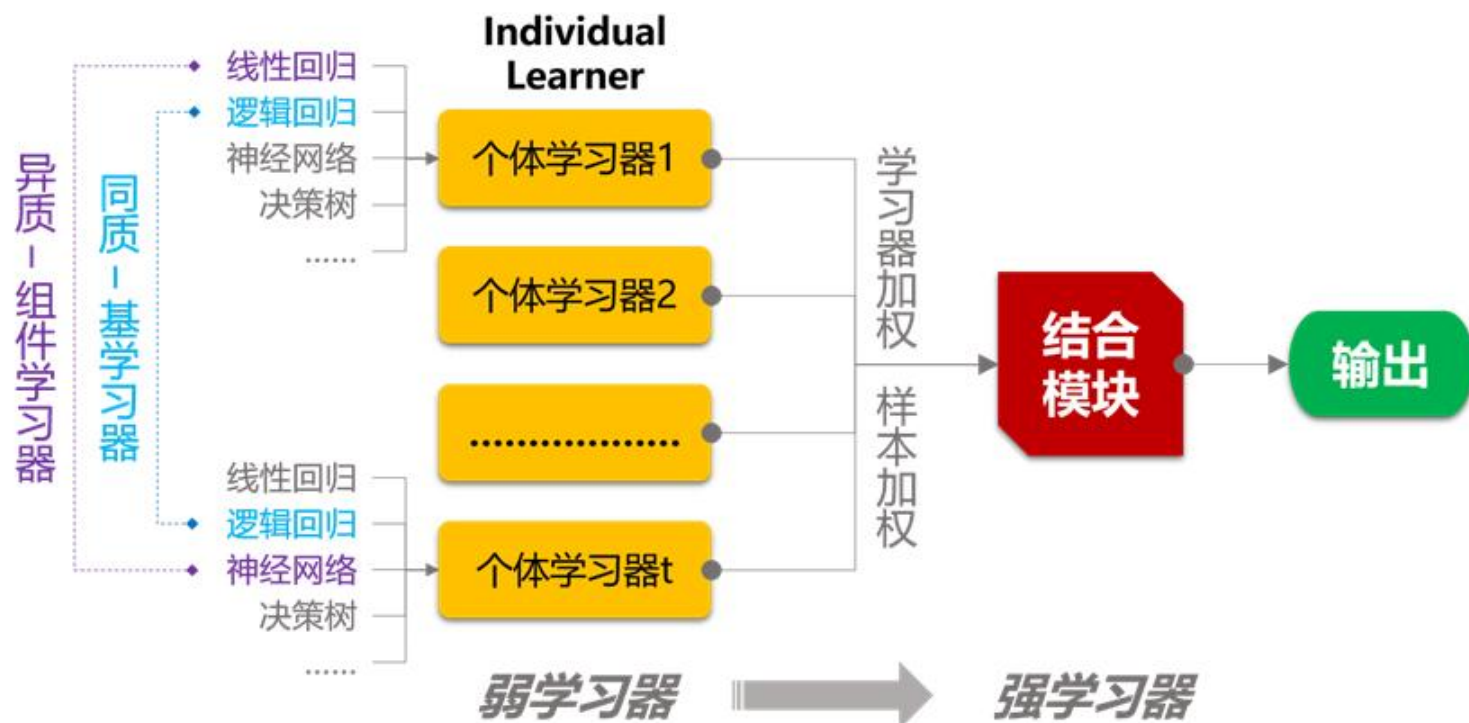


4

Bagging

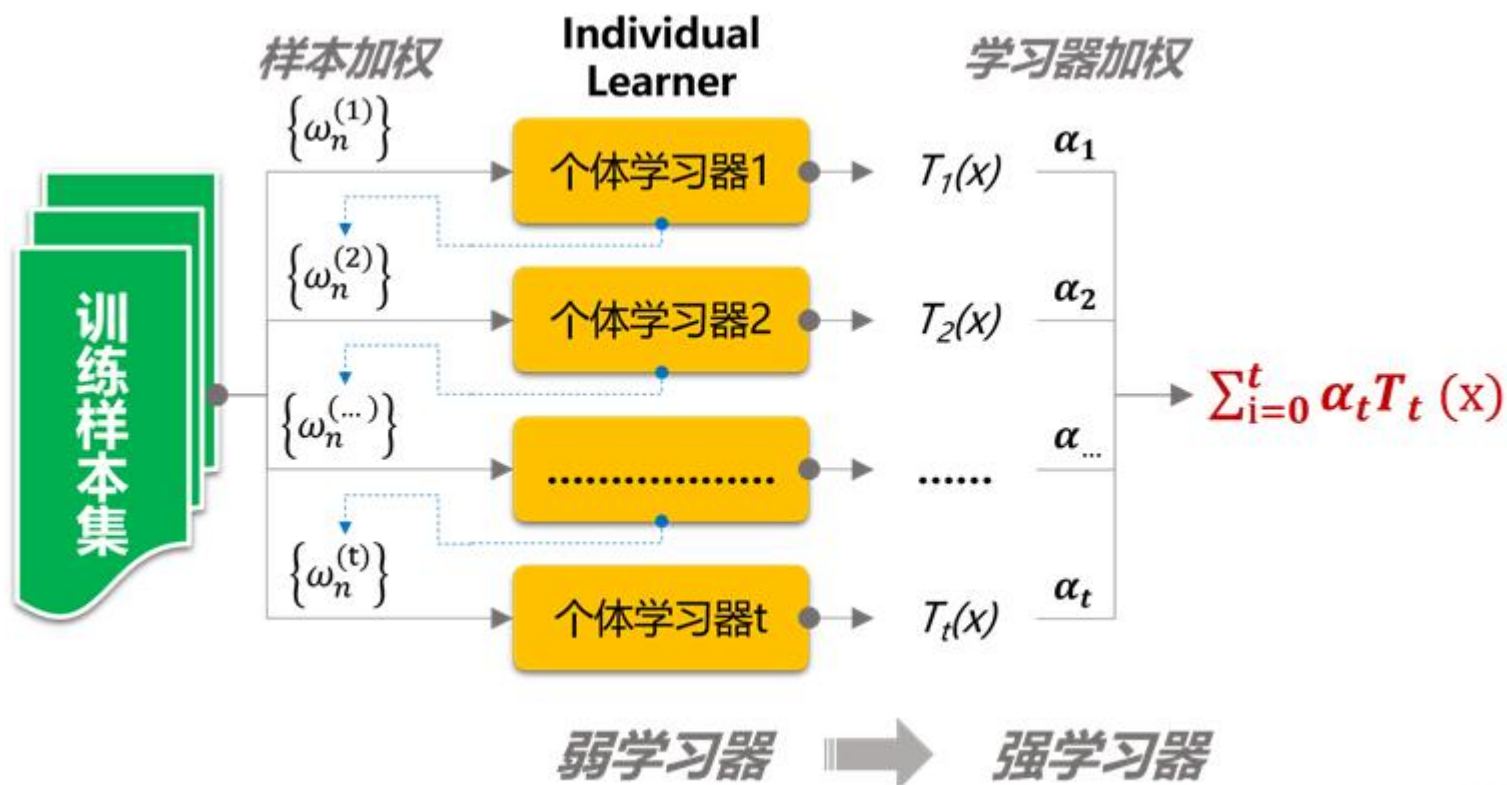
集成学习

- 集成学习将多个弱学习器进行结合，通过对样本加权、学习器加权，获得比单一学习器显著优越的泛化性能的强学习器的学习方法（Boosting和Bagging）



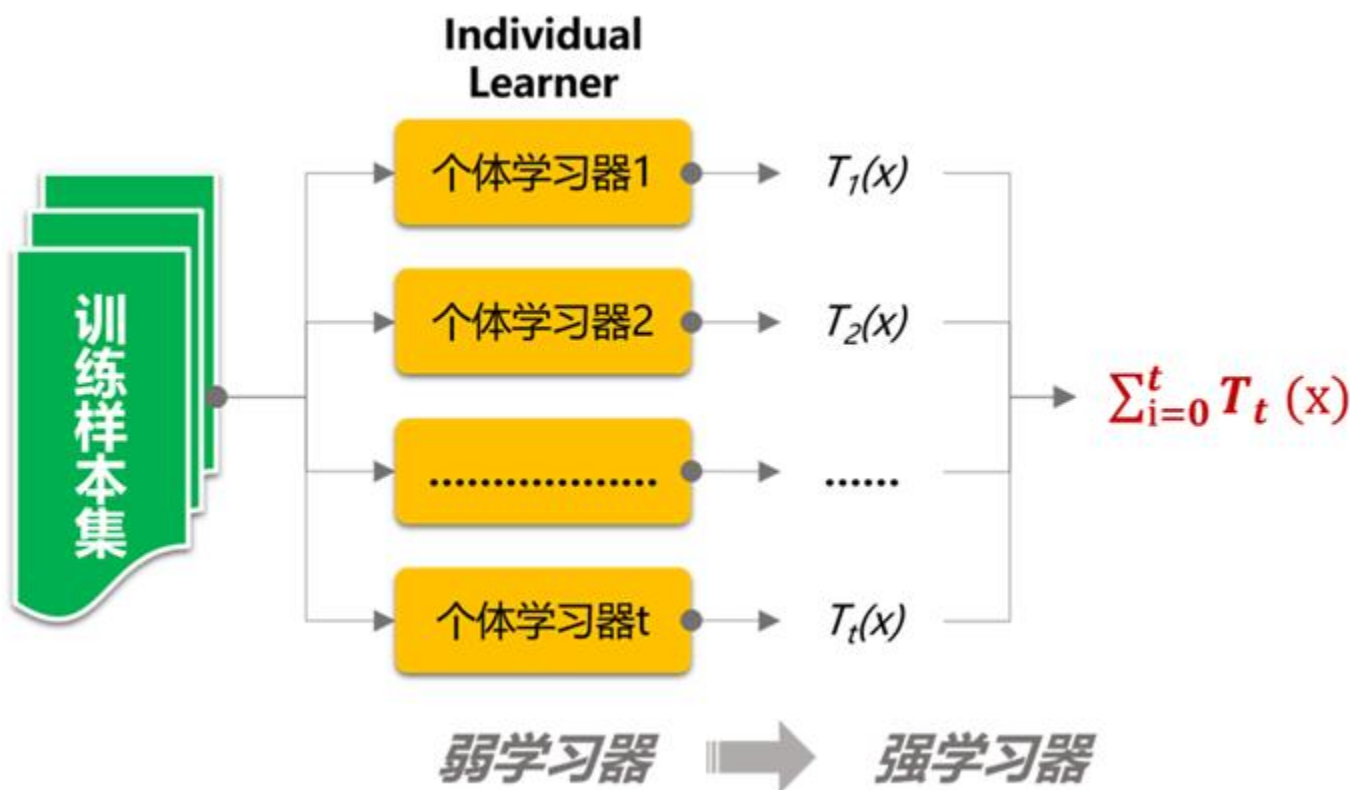
Boosting

- 个体学习器间存在强依赖关系、必须串行生成的序列化方法



Bagging

- 个体学习器间不存在强依赖关系、可同时生成的并行化方法



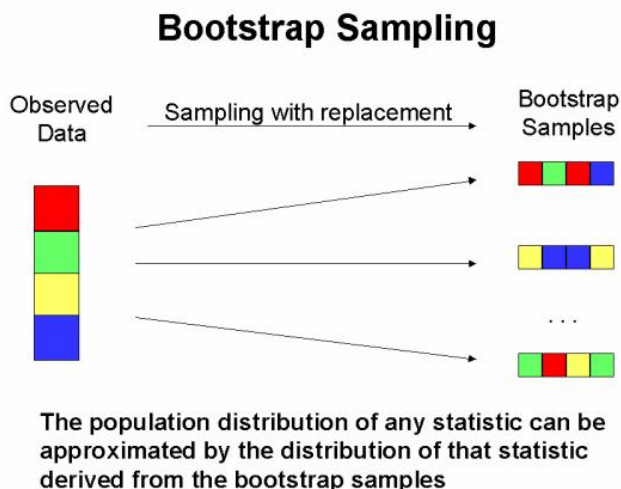
Bagging

- ❑ Bagging是一种集成学习算法，设计用来改进统计分类与回归中机器学习算法的稳定性和精确度。它也用来降低方差以帮助避免过拟合（但是不会降低偏差，高偏差模型不适用Bagging）
- ❑ Bagging通常应用在决策树算法中，但是它也可以应用到其他算法中。Bagging是模型平均方法的一个特例

Bagging

□ Bagging基于自助采样法(Bootstrap Sampling), 给定包含 m 个样本的初始训练集 S , 我们进行 m 次放回采样, 得到包含 m 个样本的采样集 S'

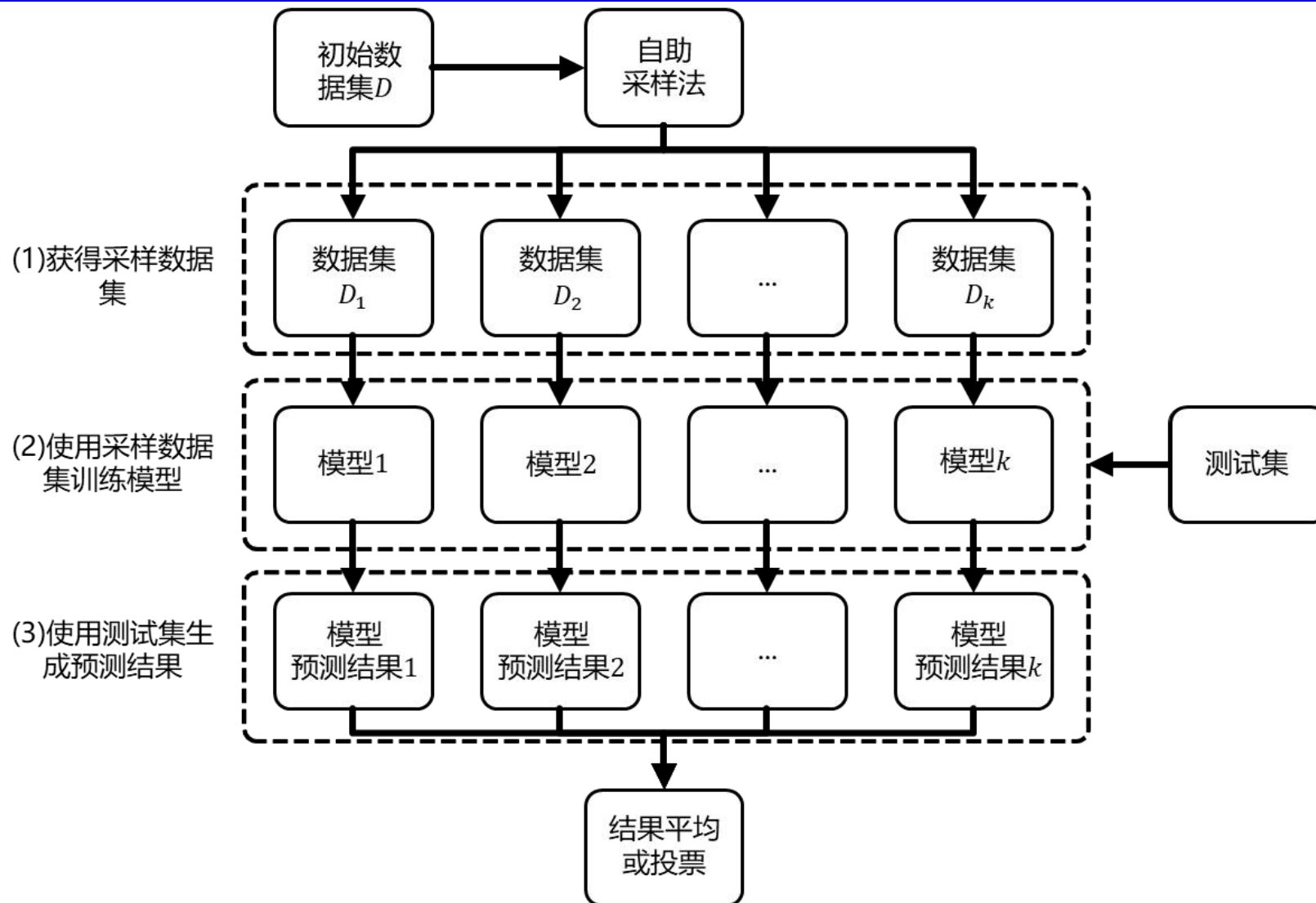
- 因此在采样数据集中会缺少一些来自原始训练集中的样本, 同时也会包含一些被多次选中重复的样本



Bagging

- ❑ 基于自助采样法构造 k 个含有 m 个训练样本的采样集，并在每个采样集上进行训练得到 k 个模型
- ❑ 每个采样集之间所含样本的差异会导致 k 个模型之间存在差异，在测试集上产生的误差就不同，而这也是Bagging方法能够起作用的原因
- ❑ 如果是分类算法，则 k 个模型投出最多票数的类别或者类别之一为最终类别。如果是回归算法，对 k 个模型得到的回归结果进行算术平均得到的值为最终的模型输出

Bagging



Bagging分析

- 假设构造的第 i 个模型在每个样本上的误差是 ϵ_i ，并且 ϵ_i 服从均值为零、方差为 $\mathbb{E}[\epsilon_i^2] = v$ 且协方差为 $\mathbb{E}[\epsilon_i, \epsilon_j] = c$ 的多维正态分布
- 所有模型的平均预测误差的平方是

$$\begin{aligned} E \left[\left(\frac{1}{k} \sum_i \epsilon_i \right)^2 \right] &= \frac{1}{k^2} E \left[\sum_i \left(\epsilon_i^2 + \sum_{j \neq i} \epsilon_i \epsilon_j \right) \right] \\ &= \frac{1}{k} v + \frac{k-1}{k} c \end{aligned}$$

Bagging分析

- ❑ 如果误差完全相关 ($c = v$)，均方误差减少到 v ，因此模型平均没有任何帮助
- ❑ 在误差完全不相干 ($c = 0$) 的情况下，该集成平方误差的期望即为 v/k
- ❑ 因此，集成平方误差的期望会随着集成规模增大而线性减小。集成至少会和它的任何成员表现得一样好，并且如果成员的误差是独立的，集成将显著地比其成员表现得更好

Bagging分析

- 神经网络能找到足够多的不同的解，意味着他们可以从模型平均中受益(即使所有模型都在同一数据集上训练)
- 神经网络中随机初始化的差异、小批量的随机选择、超参数的差异或不同输出的非确定性实现往往足以使得集成中的不同成员具有部分独立的误差

谢谢！



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY